Exercițiu: Verificatorul stării site-urilor

Oct 13, 2026 · @Emil Cebotaros

Scenariu

Vei construi un instrument mic care verifică dacă o listă de site-uri funcționează.

Un administrator de sistem trebuie să știe care site-uri sunt active, cât de mari sunt răspunsurile și care eșuează. Fiecare parte folosește funcțiile scrise înainte, dar dacă te blochezi la o parte, treci mai departe: poți obține puncte la celelalte.

- **Total:** 100 de puncte
- **Permis:** notițele tale și documentația Python și `requests`
- **Necesar:** biblioteca `requests` și fișierul de pornire `checker.py`

Cod de pornire

Profesorul îți dă un fișier de pornire numit `checker.py`. Conține lista `SITES`, lista `SECURITY_HEADERS` și șapte funcții goale: `build_url`, `is_https`, `show_response`, `check_site`, `scan_sites`, `check_headers` și `retry_check`. Lista `SITES` conține toate adresele de care ai nevoie la examen: nu adăuga alte adrese și nu scrie adrese direct în cod. Completează funcțiile; nu schimba numele lor și nici parametrii.

Partea 1: Funcții (10 puncte)

Completează cele două funcții ajutătoare și testează-le cu `print`.

1. Completează `build_url(host, path)`. Ea trebuie să funcționeze indiferent dacă `path` începe sau nu cu `/`.
   - `build_url("example.com", "login")` returnează `"https://example.com/login"`
   - `build_url("example.com", "/login")` returnează `"https://example.com/login"`
2. Completează `is_https(url)`.
   - `is_https("https://example.com")` returnează `True`
   - `is_https("http://example.com")` returnează `False`

Testează fiecare funcție cu cel puțin 2 exemple.

Partea 2: Trimiterea unei cereri (10 puncte)

Completează `show_response(url)`: ea folosește biblioteca `requests` ca să trimită o cerere GET la `url` și afișează două valori (încearc-o cu `https://example.com`):

- codul de stare
- valoarea antetului `Content-Type`

Exemplu de rezultat:

```
Status: 200
Content-Type: text/html; charset=UTF-8
```

Partea 3: Un verificator reutilizabil (20 puncte)

Completează `check_site(url)` astfel încât să returneze un dicționar și să nu se oprească niciodată cu eroare, orice ar face site-ul.

Exemplu de rezultat pentru un site care merge:

```
{"status": 200, "content_type": "text/html", "length": 1256, "server": "ECS"}
```

Cerințe:

- Folosește un timeout de 5 secunde.
- Dacă site-ul nu există sau este prea lent, returnează `{"status": None, "error": "<mesaj scurt>"}` în loc să te oprești cu eroare.
- Dacă antetul `Server` lipsește, folosește `"unknown"` ca valoare.

Testează funcția pe fiecare adresă din `SITES`.

Partea 4: Verificarea mai multor site-uri (20 puncte)

Completează `scan_sites()`, care nu primește parametri. Ea verifică fiecare site din `SITES`, afișează doar pe cele care merg și tipărește un rezumat.

1. Apelează `check_site` pentru fiecare adresă din `SITES` și păstrează rezultatele într-o listă.
2. Afișează doar site-urile care au returnat statusul `200`.
3. Sortează-le după `length`, cel mai mare primul.
4. La final, afișează câte site-uri au mers și câte au eșuat.

Exemplu de rezultat cu lista `SITES` din fișierul de pornire (lista de mai jos este scurtată; numerele tale vor diferi, iar adresa `/status/200,500` răspunde aleatoriu, deci totalurile pot varia cu 1):

```
https://httpbin.org/html  -> 200, 3741 octeți
https://httpbin.org/json  -> 200, 429 octeți
https://example.com       -> 200, 1256 octeți
...
Reușite: 9 | Eșuate: 4
```

Partea 5: Audit de securitate (20 puncte)

Securitatea unui site depinde și de anteturile pe care le trimite serverul. Patru dintre cele mai importante sunt în lista `SECURITY_HEADERS` din fișierul de pornire:

Antet	La ce folosește
`Strict-Transport-Security`	Obligă browserul să folosească doar HTTPS
`Content-Security-Policy`	Limitează de unde poate încărca pagina scripturi și resurse
`X-Frame-Options`	Împiedică includerea paginii în alt site (clickjacking)
`X-Content-Type-Options`	Oprește browserul să ghicească tipul conținutului

Completează `check_headers(url)`. Ea trimite o cerere GET (timeout de 5 secunde) și verifică care dintre antetele din `SECURITY_HEADERS` apar în răspuns. Rezultatul este un dicționar ca acesta:

```
{
    "present": ["X-Frame-Options"],
    "missing": ["Strict-Transport-Security", "Content-Security-Policy", "X-
Content-Type-Options"],
    "score": 25,
}
```

Cerințe:

- `present` și `missing` sunt liste și păstrează ordinea din `SECURITY_HEADERS`.
- `score` este procentul antetelor prezente, ca număr întreg (1 din 4 înseamnă 25, 2 din 4 înseamnă 50, toate înseamnă 100).

- Numele antetelor nu țin cont de majuscule: `x-frame-options` și `X-Frame-Options` sunt același antet.
- Dacă cererea eșuează, returnează `{"status": None, "error": "<mesaj scurt>"}`.

Puncte: forma rezultatului (3), pagină fără antete (3), potrivirea indiferent de majuscule și ordinea (5), toate antetele prezente (3), scorul (3), site inaccesibil (3).

Partea 6: Încercări repetate (20 puncte)

Serverele au uneori defecțiuni de moment: o eroare 5xx sau o conexiune picată nu înseamnă că site-ul este mort. Completează `retry_check(url, attempts=3)`, care încearcă din nou, de cel mult `attempts` ori.

Reguli:

- Fiecare încercare este o cerere GET cu timeout de 5 secunde.
- Dacă încercarea eșuează (conexiune, timeout) sau serverul răspunde cu un status de 500 sau mai mare, mai încerci, până la `attempts` încercări în total.
- Dacă serverul răspunde cu un status sub 500 (inclusiv 404), te oprești imediat: o eroare 4xx nu dispare dacă încerci din nou.
- Nu aștepta între încercări.
- Numărul de încercări raportat trebuie să fie egal cu numărul de cereri trimise cu adevărat.

Funcția returnează un dicționar cu `status` (ultimul status primit, sau `None`) și `attempts` (câte încercări ai făcut). Dacă toate încercările eșuează cu o eroare de conexiune, dicționarul conține și cheia `"error"`.

```
retry_check("https://httpbin.org/status/200")    # {"status": 200,
"attempts": 1}
retry_check("https://httpbin.org/status/404")    # {"status": 404,
"attempts": 1}
retry_check("https://httpbin.org/status/500")    # {"status": 500,
"attempts": 3}
retry_check("https://httpbin.org/status/500", attempts=2)   # {"status": 500,
"attempts": 2}
retry_check("http://this-site-does-not-exist.invalid")
# {"status": None, "error": "...", "attempts": 3}
```

Dovada: main.py

Pe lângă `checker.py`, creează în același folder un fișier `main.py` care dovedește că fiecare parte funcționează. Profesorul va rula `python main.py` și va citi ce afișează.

Pornește de la acest schelet și completează-l cu apelurile tale:

```
from checker import build_url, is_https, show_response, check_site,
scan_sites, check_headers, retry_check

# Partea 1: apeluri pentru build_url și is_https

# Partea 2: apeluri pentru show_response

# Partea 3: apeluri pentru check_site

# Partea 4: apelul scan_sites()

# Partea 5: apeluri pentru check_headers

# Partea 6: apeluri pentru retry_check
```

Cerințe:

- Apelează fiecare funcție care primește parametri de cel puțin două ori: un caz care merge și un caz limită sau care eșuează (de exemplu `is_https("http://example.com")`, `check_site` pe o adresă inexistentă, `retry_check` pe `/status/404`). Pe `scan_sites()` o apelezi o dată.
- Afișează fiecare rezultat cu o etichetă, ca să poată fi citit, de exemplu `print("Partea 1, build_url:", build_url("example.com", "login"))`. Funcția `show_response` își afișează singură rezultatul.
- Nu copia funcțiile în `main.py`: importă-le din `checker`.
- `python main.py` trebuie să ruleze de la început până la sfârșit, fără să se oprească cu eroare.
- Dacă o parte nu funcționează, lasă apelul în `main.py` și scrie un comentariu despre ce nu merge.

Ce trebuie predat

Predă un folder cu numele tău, care conține două fișiere: `checker.py` cu Părțile 1 până la 6 funcționale și `main.py`, care dovedește că fiecare parte funcționează (vezi secțiunea "Dovada: main.py").

Notare

Codul care funcționează aduce 60% din punctajul fiecărei părți; gestionarea erorilor și curățenia codului împart restul de 40%. Profesorul rulează `python main.py` ca să vadă că fiecare parte funcționează.

Criteriu	Pondere
Codul funcționează și dă rezultatul corect (dovedit în main.py)	60%
Erorile sunt gestionate (programul nu se oprește)	20%
Nume clare, structură și comentarii	20%

Partea	Puncte
1. Funcții	10
2. Trimiterea unei cereri	10
3. Un verificator reutilizabil	20
4. Verificarea mai multor site-uri	20
5. Audit de securitate	20
6. Încercări repetate	20
Total	**100**